Deston Willis

Dr. Forouraghi

Research Project

15 April 2026

Report 3

The first thing I did was create a file uploader in the sidebar for users to upload a JSON file containing the rubric for the student's code. While I started with having the app accept pdf documents, I decided to switch to JSON since it was easier for the model to understand and use the rubric compared to a pdf. When a file is uploaded, it is converted into text so that it can be given to the model when the app is invoked. This is the code for the rubric file uploader:

```
# Sidebar for Reset button and rubric upload
with st.sidebar:
    st.title("Upload Grading Rubric")
    # Uploads and stores a JSON rubric
    rubric_file = st.file_uploader(label="Please upload a JSON file containing the grading rubric", type=["json"])
    if rubric_file:
        rubric_data = json.load(rubric_file)
        rubric_text = json.dumps(rubric_data, indent=2)
        st.success("Grading rubric uploaded")
    # Reset button
    st.title("Reset Chat History")
    st.button("Reset", on_click=reset_button)

# Chat Window
# Creates a place for users to upload files and a button to start the model's evaluation
file_input = st.file_uploader(label="Please Upload a Java File to be evaluated", type=["java"])
if file_input:
    st.session_state.chat_history.append(AIMessage("Files Uploaded: " + file_input.name))
    if not rubric_text:
        st.warning("There is no rubric.")
    else:
        st.button("Evaluate Code", on_click=run_model_button)
```

To allow the agent to detect and fix syntax errors, I significantly changed the agent node and the LangGraph as a whole. I created phases of the grading process so that the model knows what to do instead of just guessing what to do. The first phase is compile_original, which is where the original code is executed to see if it runs or not. The next phase is fix_syntax, which

will only happen if the code doesn’t compile and run successfully. In this phase the model will attempt to fix any syntax errors and compile and run the new code. The next phase is create_tests, which is where the model will create a test_harness which contains code that tests the student’s code using inputs and expected outputs that are listed in the rubric. The last phase is final, which is where the model will take all of the test results and outputs from the compiler tool and provide a grade based on the rubric. To achieve this I have 3 nodes and two conditional edge functions that make up the LangGraph. The first node is the call_model node. This node contains a system prompt which describes the workflow that the model has to follow as well as provide data such as the phase and the code that the agent will need to determine how to respond. This is the code for the call_model node:

```python
# State Class is defined
class AgentState(TypedDict):  6+ usages  new *
    # Add_messages is a specialized reducer that appends new messages to history
    messages: Annotated[list[BaseMessage], add_messages]
    rubric: str
    code: str
    compiler_runs: int
    last_compile_result: str
    phase: str
    working_code: str
    test_harness_code: str
    compiler_results: list[dict]

def call_model(state: AgentState):  1 usage  new *
    """
    The 'Thought' Node.
    The model follows the designated workflow to go through the required phases and call tools when neccesary.
    """

    # Get state variables
    messages = state['messages']
    rubric = state['rubric']
    code = state['code']
    compiler_runs = state.get('compiler_runs', 0)
    last_compiler_result = state.get('last_compiler_result', "")
    phase = state.get('phase', "compile_original")
    working_code = state.get('working_code') or code

    # System prompt that describes required workflow
    system_prompt = SystemMessage(
        content=(
            "You are a grading assistant. Follow this workflow exactly:\n"
            "In the compile_original phase call the java_compiler tool with the original submitted code exactly as it is with no changes.\n"
            "If the original compile result is false, fix only the syntax errors that prevent the code from compiling. Do not fix any logical or formatting errors. "
            "Then call the java_compiler tool with the fixed code. \n "
            "Once there is working code that compile sucessfully, create test harness code that uses the inputs as parameters/test values and compares the expected outputs in the rubric with the actual output."
            "The test harness should print each test's input, expected output, actual output, and Pass/Fail. \n"
            "Call java_compiler with the test_harness code. The final grade must be based on the code's compilation and whether the test harness' actual outputs match the expected outputs in the rubric.\n"
            "Return: syntax error count and syntax errors present based only on the original submitted code's first compile result. "
            "Then include fixed syntax errors, test results, rubric score, and feedback. The test results must include the actul output of the code. Do no generate output values."
            "Mention logical issues only as grading feedback, not as fixes.\n"
            "Grade only using point values and categories explicitly present in the rubric. Report the earned points for each rubric section and the final total earned score. "
            "You must use this equation to calculate final grade. Final grade equation: (Comilation Score + All Test Scores) / Total Possible Points \n"
            "Note that the java_compiler tool returns JSON with compiled, ran, output, and error fields.\n"
            "If the message history already contains java_compiler output, continue from that output instead of restarting the workflow.\n"
            f"Current phase: {phase}\n"
            f"Compiler runs already completed: {compiler_runs}. The maximum allowed is 3.\n"
            f"Most recent compiler result:\n{last_compiler_result or 'No compiler result yet.'}\n"
            f"Rubric:\n{state['rubric']}\n"
            f"Original submitted code:\n{state['code']}\n"
            f"Current working code:\n{working_code}"
        )
    )

    response = llm.invoke([system_prompt] + messages)
    return {"messages": [response]}
```

After this is the should_continue conditional edge. This edge will route to the call_tool node if the last message is a tool call and route to the final_response node if there was not a tool call. Since a lot of times the model would try to call the tool but instead of responding with a proper tool call it would have the tool call in a JSON format, I added functionality in this edge and throughout the call_tool node that allows for these JSON tool calls to be detected and used as well to call the java_compiler tool. This is the code for the should_continue edge:

```python
def should_continue(state: AgentState):  1 usage  new *
    """
    The Conditional Edge.
    Checks if the last message contains tool calls.
    """
    last_message = state['messages'][-1]
    # Returns final node if the graph has looped too many times
    if state['compiler_runs'] >= 10:
        return "final"
    # Returns tool if the message is a tool call
    if last_message.tool_calls:
        return "tool"
    # Retuns tool if the message is a JSON response that contains the tool call
    if isinstance(last_message, AIMessage):
        text = last_message.content.strip()
        if text.startswith("```"):
            text = re.sub( pattern: r"^```(?:json)?\s*",  repl: "", text)
            text = re.sub( pattern: r"\s*```$",  repl: "", text)
        try:
            json_message = json.loads(text)
            if json_message.get("name") == "java_compiler":
                return "tool"
        except json.JSONDecodeError:
            pass
    # Return final if there is no tool call
    return "final"
```

The next node is the call_tool node. I am using a custom tool node instead of the built in one so that the tool can be called when the model responds with tool calls in JSON messages as well as so the node can use the output from the tool to control the phases of the grading process. In order for the call_tool node to have the proper information to control the phases, I added a couple more variables to the output of the java_compiler tool. Specifically, in addition to returning the output/error of the code, I also return whether the code compiled and ran successfully or not. This ensures that the model moves to the correct phase depending on if the code compiles and runs or not. Once the phase is set for the model, this node will create a

message that contains the tool output and if the phase is create_tests, a message that will prompt the model to create a test harness that will contain any tests that are described in the rubric as well so that the call_model can use that code for its tool call when the graph loops back to it. This is the code for the call_tool node:

```
def call_tool(state: AgentState):  1 usage  new *
    """
    The Tool Node.
    Runs the compiler tool, stores the output, and controls phases.
    """
    last_message = state['messages'][-1]
    compiler_runs = state.get('compiler_runs', 0)
    phase = state['phase']
    next_phase = ""
    tool_call_id = f"java_compiler_{compiler_runs + 1}"
    structured_message = False
    working_code = state['working_code'] or state['code']
    test_harness_code = state.get('test_harness_code', "")
    compiler_results = list(state.get('compiler_results', []))

    # Calls tool if last message is a tool call
    if last_message.tool_calls:
        tool_call = last_message.tool_calls[0]
        arguments = tool_call.get("args", {})
        tool_call_id = tool_call.get("id", tool_call_id)
        structured_message = True
    # Calls tool if the last message is a JSON response with the tool call by extracting the tool call from it
    else:
        text = last_message.content.strip()
        if text.startswith("```"):
            text = re.sub( pattern: r"^```(?:json)?\s*",  repl: "", text)
            text = re.sub( pattern: r"\s*```$",  repl: "", text)
        # Throws error if JSON isn't formatted correctly
        try:
            payload = json.loads(text)
            arguments = payload.get("arguments", {})
        except json.JSONDecodeError:
            arguments = {}

    code = arguments.get( key: "code", state['working_code'] or state['code'])
    result = java_compiler.invoke({"code": code})
    # Throws error if JSON is invalid
    try:
        compile_result = json.loads(result)
    except json.JSONDecodeError:
        compile_result = {}
```

```python
# Adds tool output to compiler results
compiler_results.append(
    {
        "run_number": compiler_runs + 1,
        "phase": phase,
        "code": code,
        "result": compile_result
    }
)

# Controls phases so that the LLM knows what to do
if phase == "compile_original":
    if compile_result.get("compiled"):
        next_phase = "create_tests"
        working_code = code
    else:
        next_phase = "fix_syntax"
elif phase == "fix_syntax":
    if compile_result.get("compiled"):
        next_phase = "create_tests"
        working_code = code
    else:
        next_phase = "final"
        working_code = code
elif phase == "create_tests":
    next_phase = "final"
    test_harness_code = code
```

```python
# Used for regular tool call
if structured_message:
    tool_result_message = ToolMessage(
        content=result,
        tool_call_id=tool_call_id,
        name="java_compiler",
    )
    result_message = HumanMessage(content=(
        f"Next phase: {next_phase}. If the next phase is create_tests, create a test harness by calling the java compiler tool that uses only the rubric's inputs as parameters/test values "
        "and compares actual outputs to the expected outputs in the rubric. Only use expected outputs listed in the rubric."
        "All inputs and expected outputs must be from the rubric. To do this, you must replace put the test harness code in the main method instead of its own class. "
        "If there is a declared variable, replace it with the test inputs."
        )
    )
# Used for tool calls that were in JSON response
else:
    tool_result_message = HumanMessage(content=(
        "java_compiler output:\n"
        f"{result}\n"
        f"Next phase: {next_phase}. If the next phase is create_tests, create a test harness by calling the java compiler tool that uses only the rubric's inputs as parameters/test values "
        "and compares actual outputs to the expected outputs in the rubric. Only use expected outputs listed in the rubric. "
        "All inputs and expected outputs must be from the rubric. To do this, you must replace put the test harness code in the main method instead of its own class. "
        "If there is a declared variable, replace it with the test inputs."
        )
    )

# Used for regular tool calls
if structured_message:
    return {
        "compiler_runs": compiler_runs + 1,
        "last_compile_result": result,
        "phase": next_phase,
        "working_code": working_code,
        "test_harness_code": test_harness_code,
        "compiler_results": compiler_results,
        "messages": [tool_result_message + result_message]
    }
# Used for tool calls that were in JSON response
else:
    return {
        "compiler_runs": compiler_runs + 1,
        "last_compile_result": result,
        "phase": next_phase,
        "working_code": working_code,
        "test_harness_code": test_harness_code,
        "compiler_results": compiler_results,
        "messages": [tool_result_message]
    }
```

The next conditional edge is tool_should_continue. This function simply routes the graph back to the agent unless there have been too many compiler runs or the phase is final. In these cases, the graph is routed to the final_response node. This is the code:

```python
def tool_should_continue(state: AgentState):  1 usage  new *
    """
    The Conditional Edge.
    Checks if the phase is final or not.
    """
    compiler_runs = state.get("compiler_runs", 0)
    phase = state['phase']
    # Returns final node if the compiler has run at least 10 times or the phase is final. Otherwise, this edge loops back to agent
    if compiler_runs >= 10:
        return "final"
    if phase == "final":
        return "final"
    else:
        return "agent"
```

The last node is the final_response node. This node contains a system prompt that takes all of the compiler results and outputs from all of the test and syntax checks and produces a final

grade as well as grade breakdown and feedback based on the provided rubric. The node then invokes the model with this prompt and returns the response with all of these things. This is the code:

```
def final_response(state: AgentState):  1 usage  new *
    """
    The Final Response Node.
    Produces the final grade and the grade breakdown.
    """
    # Prompt that produces the response that contains the final grade and point distribution
    system_prompt = SystemMessage(content=(
        "Give the final grade now. Do not call tools and do not output a tool call.\n"
        "Your response must be normal text with a grade. \n"
        "Use the compiler results and the test harness output. \n"
        "Grade based on whether the actual outputs from the tests_harness code match the expected outputs in the rubric. \n"
        "If syntax errors were fixed, list them. If the test harness didn't run because compilation failed, give a grade of 0 for the tests. \n"
        "The syntax errors count and list must be based only on the original submitted code's first compile result, which is the compiler_results item with phase compile_original. \n"
        "For test inputs, report the exact input values as written in the rubric. Use only point values and grading categories explicitly present in the rubric. \n"
        "You must include the actual earned score and list each rubric section with points earned and points possible, then list the final total earned points and total possible points. \n"
        "Strict grading rules:\n"
        "- For every section, first quote or copy the exact rubric criteria text being used.\n"
        "- For every section, copy the exact point value, deduction, and category from the rubric.\n"
        "- Do not give partial credit unless the rubric explicitly says to give partial credit.\n"
        "- Syntax errors count and list should only be based on compile_original code. \n"
        "- Don't count or list syntax errors in test harness code. \n"
        "- Take off 10 points in compilation score if there are any syntax errors in the compile_original code.\n"
        "- Compilation score must not be based on final code. It must be based on the original code.\n"
        "- Final grade must be calculated with this equation: (Compilation Score + All test scores) / Total possible points. \n"
        "- When listing final grade, list calculation, total points, and the percentage. \n"
        "- Passed test receive all points while failed tests receive 0 points.\n"
        "- Don't pass tests if they don't have actual output.\n"
        "- Do not make up scores. Don't add points for any outputs that aren't tests.\n"
        "- You must explicitly sum all earned points step by step before stating the final grade."
        "Return: syntax feedback that contains syntax error count in the original code and a list of each syntax error that was fixed, test feedback that contains pass/fail, "
        "expected output and actual output for each test, grade breakdown that contains earned/possible points for each rubric section and the compilation section score is reduced by 10 if there are any syntax errors, "
        "final grade that is Compilation score - 10 if there are any syntax errors counted in the original code + All Test scores, and feedback.\n"
        f"Rubric:\n{state['rubric']}\n"
        f"Original submitted code:\n{state['code']}\n"
        f"Working code used before harness:\n{state.get('working_code', '')}\n"
        f"Test harness code:\n{state.get('test_harness_code', '')}\n"
        f"Compiler runs completed: {state.get('compiler_runs', 0)}\n"
        f"Final phase: {state.get('phase', '')}\n"
        f"Most recent compiler result:\n{state.get('last_compile_result', '')}\n"
        f"All compiler results:\n{json.dumps(state.get('compiler_results', []), indent=2)}"
        )
    )
    response = final_llm.invoke([system_prompt])
    return {"messages": [response]}
```

My graph has 3 nodes, two conditional edges after the agent and the tool node, and 1 edge that connects the final response to the end. This is the decoration of the graph's nodes and edges::

```
# Construct the Graph
workflow = StateGraph(AgentState)

# Define the nodes in the workflow
workflow.add_node("agent", call_model)
workflow.add_node("tool", call_tool)
workflow.add_node("final", final_response)

# Define the logic flow
workflow.set_entry_point("agent")

workflow.add_conditional_edges( source: "agent", should_continue)
workflow.add_conditional_edges( source: "tool", tool_should_continue)
workflow.add_edge( start_key: "final", END)

# Compile into a runnable application
app = workflow.compile()
```

After I set up the code for the Streamlit app, I created a rubric and 2 test files that test the model's response when the code is correct and code that has a small syntax error and fails one of the tests. The rubric is for an assignment that asks the student to create a function that takes a double and returns the doubled absolute value of the number rounded. The code is graded on whether it compiles and runs without syntax errors and if it passes the 4 tests that are laid out in the rubric. These 4 inputs for the tests are a positive number, a negative number, a positive number that should be rounded down, and a positive number that should be rounded up. Any failed tests are given 0 points. As well, if there are any syntax errors, the student is penalized 10 points in the compilation section. The first code (Test_File5) has correct code and no syntax errors. The second code (Test_File6) has a couple of syntax errors as well as missing the rounding for the number so it should fail the 3rd and 4th test. This is the results of these tests:

Rubric:

```
{
  "criteria": [
    {
      "name": "Compilation",
      "points": 40,
      "description": "The original code compiles and runs without errors. 10 points are deducted from compilation if there are any syntax error in the original code",
      "actual code output": ""
    },
    {
      "name": "Test 1",
      "points": 15,
      "description": "The code with the specified input returns the expected output",
      "test":
        {
          "input": 5,
          "expected_output": 10.0
        },
      "actual code output": ""
    },
    {
      "name": "Test 2",
      "points": 15,
      "description": "The code with the specified input returns the expected output",
      "test":
        {
          "input": -2,
          "expected_output": 4.0
        },
      "actual code output": ""
    },
```

```
    {
      "name": "Test 3",
      "points": 15,
      "description": "The code with the specified input returns the expected output",
      "test":
        {
          "input": 3.6,
          "expected_output": 7.0
        },
      "actual code output": ""
    },
    {
      "name": "Test 4",
      "points": 15,
      "description": "The code with the specified input returns the expected output",
      "test":
        {
          "input": 7.8,
          "expected_output": 16.0
        },
      "actual code output": ""
    },
    {
      "name": "Final Grade",
      "points": 100,
      "description": "This is the total points based on the points from the other criteria. The final grade is out of 100 points. Calculate it by adding Compilation earned points + Test1 Earned Points + Test2 Earned Points + Test3 Earned Points + Test4 Earned Points",
      "final grade": ""
    }
  ]
}
```


Correct Code:

```java
import java.lang.Math;
public class Java_Test5 {
    public static void main(String[] args) {
        double num = 10;
        System.out.println("New Number: " + doubleNumber(num));
    }
    public static double doubleNumber(double n) {
        n = n * 2;
        return Math.round(Math.abs(n));
    }
}
```

Correct Code Grade:

Upload Grading Rubric

Please upload a JSON file containing the grading rubric

Gradin...bric 3.json
1.7KB

+

Grading rubric uploaded

Reset Chat History

Reset

Java_Test5.java
315.0B

+

Evaluate Code

Files Uploaded: Java_Test5.java

Syntax Feedback:

- Syntax error count in the original code: 0
- List of each syntax error that was fixed: None

Test Feedback:

- **Test 1:**
  - Input: 5
  - Expected Output: 10.0
  - Actual Output: 10.0
  - Result: Pass
- **Test 2:**
  - Input: -2
  - Expected Output: 4.0
  - Actual Output: 4.0
  - Result: Pass
- **Test 3:**
  - Input: 3.6
  - Expected Output: 7.0
  - Actual Output: 7.0
  - Result: Pass
- **Test 4:**
  - Input: 7.8
  - Expected Output: 16.0
  - Actual Output: 16.0
  - Result: Pass

Grade Breakdown:

- **Compilation:** 40/40 (No syntax errors)
- **Test 1:** 15/15 (Pass)
- **Test 2:** 15/15 (Pass)
- **Test 3:** 15/15 (Pass)
- **Test 4:** 15/15 (Pass)

Final Grade:

- Calculation: (40 + 15 + 15 + 15 + 15) / 100
- Total Points: 100
- Percentage: 100%

The final grade is **100/100**.

Incorrect Code:

```java
import java.lang.Math;
public class Java_Test6 {
    public static void main(String[] args) {
        double num = 10;
        System.out.println("New Number: " + doubleNumber(num));
    }
    public static double doubleNumber(double n) {
        n = n * 2
        return Math.abs(n)
    }
}
```

Incorrect Code Grade:

Upload Grading Rubric

Please upload a JSON file containing the grading rubric

Gradin...bric 3.json
1.7KB

+

Grading rubric uploaded

Reset Chat History

Reset

AI Grading Assistant

Please Upload a Java File to be evaluated

Java_Test6.java
301.0B

+

Evaluate Code

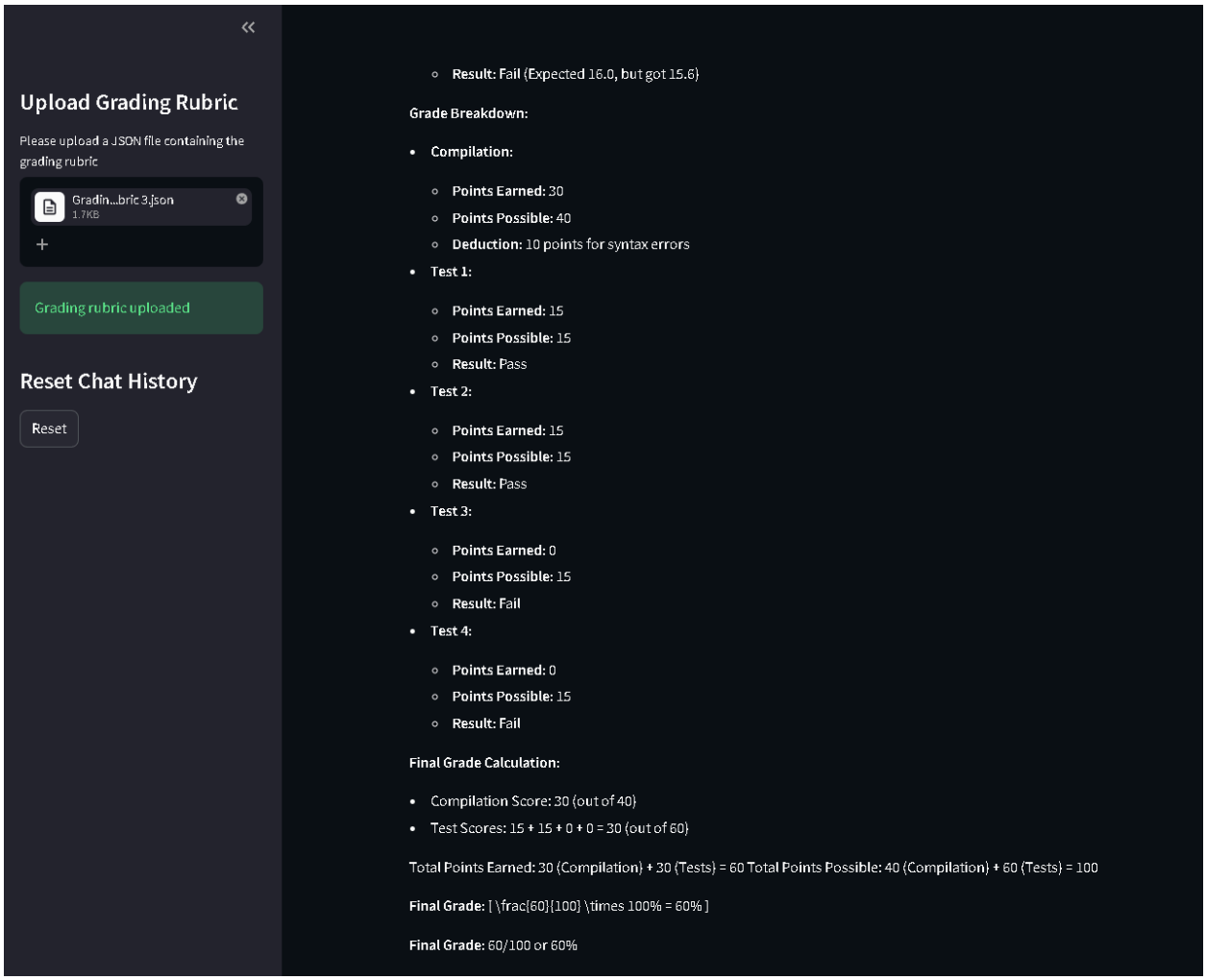
Files Uploaded: Java_Test6.java

Syntax Feedback:

- **Syntax Error Count:** 2
- **Fixed Syntax Errors:**
  - Missing semicolon at the end of line 8 (`n = n * 2`)
  - Missing semicolon at the end of line 9 (`return Math.abs(n)`)

Test Feedback:

- **Test 1:**
  - **Input:** 5
  - **Expected Output:** 10.0
  - **Actual Output:** 10.0
  - **Result:** Pass
- **Test 2:**
  - **Input:** -2
  - **Expected Output:** 4.0
  - **Actual Output:** 4.0
  - **Result:** Pass
- **Test 3:**
  - **Input:** 3.6
  - **Expected Output:** 7.0
  - **Actual Output:** 7.2
  - **Result:** Fail (Expected 7.0, but got 7.2)
- **Test 4:**
  - **Input:** 7.8
  - **Expected Output:** 16.0
  - **Actual Output:** 15.6
  - **Result:** Fail (Expected 16.0, but got 15.6)



For the correct code, the app gave it a grade of 100. This is expected since it passes all of the tests that are in the rubric and contains no syntax errors. The incorrect code was given a grade of 60/100. This also is expected since this code contained 2 syntax errors that the model was able to find and fix and the code failed the last two tests since it doesn't round the returned number.